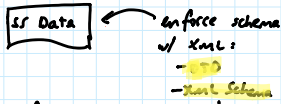


- a new "data" model. (like RM, ER, UML, OO...)
- better representation of real world.
- schema is implied by the data, rather than being declared separately.
- the most important manifestation of SSDM is **XML**.



- Roles of SSDM in DB world:

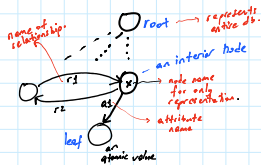
- 1) **data integration** = integration of 2 similar data w/ different schema
- 2) **sharing info on the WEB** := SSDM provides an underlying model.

- Comparison of data models:

SSDM	other (RM, ER, UML, ...)
- flexible model.	- strict model.
- better representation of real world.	- better sys. implementation " query exec. " data structures.
- offers significant advantages to users.	- efficient db programming w/ SQL.
- schemaless (self-describing)	
- schema may vary arbitrarily.	
- awkward db programming. w/ XPath, XQuery, XSLT	

- SSD representation:

- graph model with nodes & arcs. (UML is also graph model)
- Nodes: leaf or interior
- Leaf nodes hold atomic data: numeric or string. (UML nodes represent entity set)
- Interior nodes have arcs to each other representing the relationships or the name of attributes.
- Root represents entire db.



Example #1:

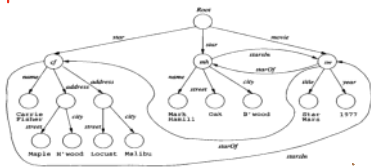
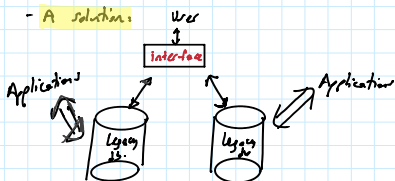


Figure 11.1: Semistructured data representing a movie and stars

- Information Integration:

- many same miniworlds
↳ w/ "similar" schemas. (but not exactly the same)
↳ w/ different model (relational, oo, ...)
- **legacy-database problem**:
- many applications depending on existing schema

- A solution:



- legacy systems are unchanged.
- semistructured data is constructed using "wrappers".
- Data is queried w/ xml programming languages (like XPath, Xquery)

11.1 Exercise:

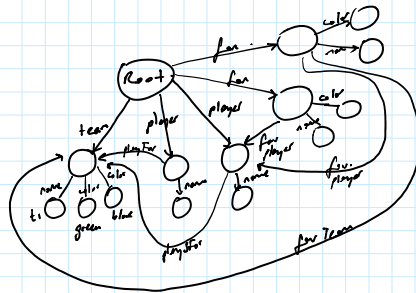
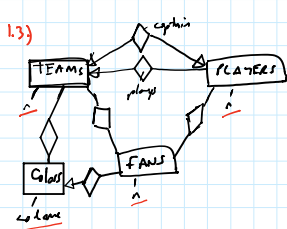
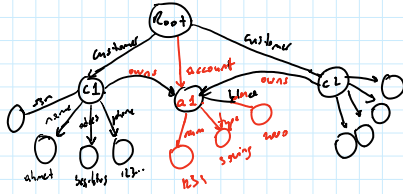
- No design schema exercises. Because SSDM does not have separate schema.
- Data organization problems.

Exercise 11.1.2: Suggest how typical data about banks and customers, as in Exercise 4.1.1, could be represented in the semistructured model.

Exercise 11.1.3: Suggest how typical data about players, teams, and fans, as was described in Exercise 4.1.3, could be represented in the semistructured model.

Exercise 11.1.4: Suggest how typical data about a geography, as was described in Exercise 4.1.6, could be represented in the semistructured model.

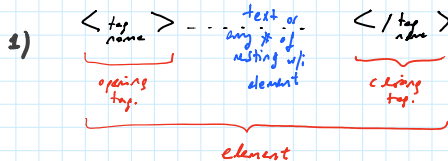
Exercise 11.1.5: UML and the semistructured-data model are both "graphical" in nature, in the sense that they use nodes, labels, and connections among nodes as the medium of expression. Yet there is an essential difference between the two models. What is it? *SSDM never represents atomic data.*



XML:

= linear form of SSDM graphs.

= tag-based notation similar to HTML. HTML talks about presentation while XML tags are intended to talk about the meaning of data



- 2) <tagname /> ⇒ no nesting.
⇒ may have attributes.

- 2 types of XML: Well-formed, Valid.

↳ Well formed = - nesting rule of tags must be obeyed.
- no predefined schema

↳ Valid XML = use whatever tags you wish.
involves a DTD that specifies allowable tags and grammar for how they may be nested.

RM → [Valid XML] → Well-formed XML
'strict schema' 'completely schemaless.'

Well-formed XML:

`<?xml version="1.0" encoding="utf-8" standalone="yes">`
`<StarMovieData>`
`<Star>`
`<Name>Carrie Fisher</Name>`
`<Address>`
`<Street>123 Maple St.</Street>`
`<City>Hollywood</City>`
`</Address>`
`<Address>`
`<Street>5 Locust Ln.</Street>`
`<City>Malibu</City>`
`</Address>`
`</Star>`
`<Star>`
`<Name>Mark Hamill</Name>`
`<Street>456 Oak Rd.</Street>`
`<City>Brentwood</City>`
`</Star>`
`<Movie>`
`<Title>Star Wars</Title>`
`<Year>1977</Year>`
`</Movie>`
`</StarMovieData>`

root
 means well-formed.

`<Movie year="1977"><Title>Star Wars</Title></Movie>`
 We could even make both child nodes be attributes by:
`<Movie title="Star Wars" year="1977"></Movie>`
 or even:
`<Movie title="Star Wars" year="1977"/>`

Figure 11.3: An XML document about stars and movies

Namespaces:

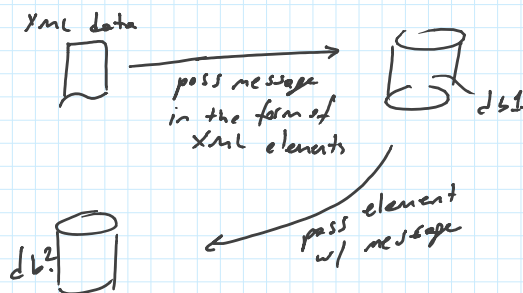
`<md:StarMovieData xmlns:md="http://.../movies">`
 name of namespace
 location of namespace.

- tags may come from different sources which may therefore have conflicting names. To distinguish different among different vocabularies, we use NAMESPACES.

- `xmlns:name="URI"` = This is an attribute which is used in a special form. In the above example, `StarMovieData` element is part of the namespace, `md`. Inside this element we may use tags of this namespace by prefixing with `md`:

XML & Databases:

- XML data is not always stored in a db.

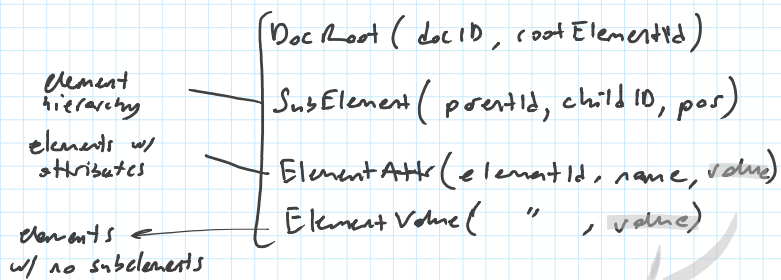


- A large XML document or streaming elements.

- 2 way to store XML data in a db:

- ↳ Store it in a parsed form.
Provide libraries (SAX, DOM) for data manipulation.
- ↳ Transform from SSOM to RM and store in a conventional way.
(Normal / JCR / not Element)

store in a conventional way.



what would the values type be? Always string?

11.2.8 Exercises for Section 11.2

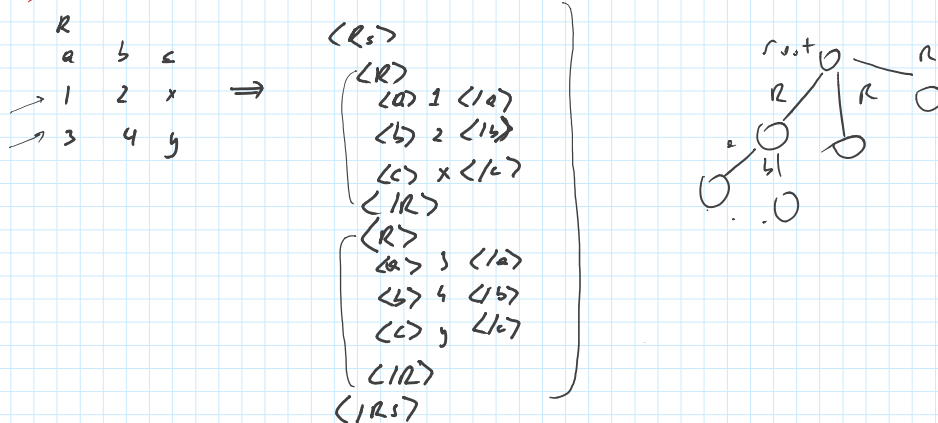
Exercise 11.2.1: Repeat Exercise 11.1.1 using XML.

Exercise 11.2.2: Show that any relation can be represented by an XML document. *Hint:* Create an element for each tuple with a subelement for each component of that tuple.

Exercise 11.2.3: How would you represent an empty element (one that had neither text nor subelements) in the database schema of Section 11.2.7?

Exercise 11.2.4: In Section 11.2.7 we gave a database schema for representing documents that do not have *mixed content* — elements that contain a mixture of text (#PCDATA) and subelements. Show how to modify the schema when elements can have mixed content.

2.2.)



DTD (doc. type def.)

general structure

```

<!DOCTYPE root-tag[
  <!ELEMENT el-name (comp')>
  :
]>
  
```

— <!ELEMENT Title (#PCDATA)>

= means no element nested within Title.
there is only text.

<!ELEMENT Foo EMPTY>

= means <Foo /> can only appear.


```

<!DOCTYPE Stars [
  <!ELEMENT Stars (Star*)>
  <!ELEMENT Star (Name, Address+, Movies)>
  <!ELEMENT Name (#PCDATA)>
  <!ELEMENT Address (Street, City)>
  <!ELEMENT Street (#PCDATA)>
  <!ELEMENT City (#PCDATA)>
  <!ELEMENT Movies (Movie*)>
  <!ELEMENT Movie (Title, Year)>
  <!ELEMENT Title (#PCDATA)>
  <!ELEMENT Year (#PCDATA)>
]>

```

Figure 11.6: A DTD for movie stars

* : ≥ 0
 + : ≥ 1
 ? : 0 or 1
 | : or used like (...|...|...)

```

<?xml version = "1.0" encoding = "utf-8" standalone = "no"?>
<!DOCTYPE Stars SYSTEM "stars.dtd">

```

```

<Stars>
  <Star>
    <Name>Carrie Fisher</Name>
    <Address>
      <Street>123 Maple St.</Street>
      <City>Hollywood</City>
    </Address>
    <Address>
      <Street>5 Locust Ln.</Street>
      <City>Malibu</City>
    </Address>
    <Movies>
      <Movie>
        <Title>Star Wars</Title>
        <Year>1977</Year>
      </Movie>
      <Movie>
        <Title>Empire Strikes Back</Title>
        <Year>1980</Year>
      </Movie>
      <Movie>
        <Title>Return of the Jedi</Title>
        <Year>1983</Year>
      </Movie>
    </Movies>
  </Star>
  <Star>
    <Name>Mark Hamill</Name>
    <Address>
      <Street>456 Oak Rd.</Street>
      <City>Brentwood</City>
    </Address>
    <Movies>
      <Movie>
        <Title>Star Wars</Title>
        <Year>1977</Year>
      </Movie>
      <Movie>
        <Title>Empire Strikes Back</Title>
        <Year>1980</Year>
      </Movie>
      <Movie>
        <Title>Return of the Jedi</Title>
        <Year>1983</Year>
      </Movie>
    </Movies>
  </Star>
</Stars>

```

Figure 11.7: Example of a document following the DTD of Fig. 11.6

valid XML

- <! ATTLIST elem_name att_name type #REQ >

```

<ELEMENT Movie EMPTY>
<ATTLIST Movie
  title CDATA #REQUIRED
  year CDATA #REQUIRED
  genre (comedy | drama | sciFi | teen) #IMPLIED
>

```

no subelements
- CDATA
- ID
- IDREF(s)
enum
optional

Figure 11.8: Data about movies will appear as attributes

<Movie title = "... " year = "... " genre = "... />

- IDs and Refs =

In O/P, we can give some attributes

the type ID

the type IDREF

the type IDREFS

no

```

<!DOCTYPE StarMovieData [
  <ELEMENT StarMovieData (Star*, Movie*)>
  <ELEMENT Star (Name, Address)*>
  <!ATTLIST Star
    starID ID #REQUIRED
    starredIn IDREFS #IMPLIED
  >
  <ELEMENT Name (#PCDATA)>
  <ELEMENT Address (Street, City)*>
  <ELEMENT Street (#PCDATA)>
  <ELEMENT City (#PCDATA)>
  <ELEMENT Movie (Title, Year)*>
  <!ATTLIST Movie
    movieID ID #REQUIRED
    starredIn IDREFS #IMPLIED
  >
  <ELEMENT Title (#PCDATA)>
  <ELEMENT Year (#PCDATA)>
]>

```

Figure 11.9: A DTD for stars and movies, using ID's and IDREF's

Stars and movies have equal status; both are subelements of the root.

```

<?xml version="1.0" encoding="utf-8" standalone="yes" ?>
<StarMovieData>
  <Star starID="1" starredIn="sw">
    <Name>Carrie Fisher</Name>
    <Address>
      <Street>123 Maple St.</Street>
      <City>Hollywood</City>
    </Address>
    <Address>
      <Street>5 Locust Ln.</Street>
      <City>Malibu</City>
    </Address>
  </Star>
  <Star starID="2" starredIn="sw">
    <Name>Mark Hamill</Name>
    <Address>
      <Street>456 Oak Rd.</Street>
      <City>Brentwood</City>
    </Address>
  </Star>
  <Movie movieID="sw" paraOf="cfham">
    <Title>Star Wars</Title>
    <Year>1977</Year>
  </Movie>
</StarMovieData>

```

Figure 11.10: Adding stars-in information to our XML document

Exercise 11.3.1: Add to the document of Fig. 11.10 the following facts:

- Carrie Fisher and Mark Hamill also starred in *The Empire Strikes Back* (1980) and *Return of the Jedi* (1983).
- Harrison Ford also starred in *Star Wars*, in the two movies mentioned in (a), and the movie *Firewall* (2006).
- Carrie Fisher also starred in *Hannah and Her Sisters* (1985).
- Matt Damon starred in *The Bourne Identity* (2002).

Exercise 11.3.2: Suggest how typical data about banks and customers, as was described in Exercise 4.1.1, could be represented as a DTD.

BANK DB

Exercise 11.3.3: Suggest how typical data about players, teams, and fans, as was described in Exercise 4.1.3, could be represented as a DTD.

SPORTS DB

Exercise 11.3.4: Suggest how typical data about a genealogy, as was described in Exercise 4.1.6, could be represented as a DTD.

GENEALOGY DB

Exercise 11.3.5: Using your representation from Exercise 11.2.2, devise an algorithm that will take any relation schema (a relation name and a list of attribute names) and produce a DTD describing a document that represents that relation.

3.2) BANK DB

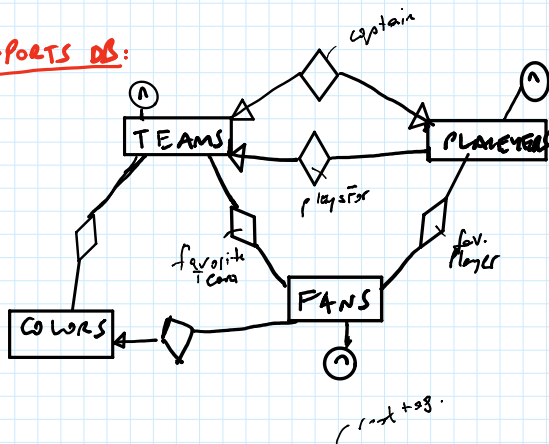


```

<!DOCTYPE Bank [
  <ELEMENT BANK (CUSTOMER*, ACCOUNT*)>
  <ELEMENT CUSTOMER (name, address, phone, ssn)>
  <!ATTLIST CUSTOMER
    custid ID,
    owns IDREFS>
  <ELEMENT ACCOUNT (num, type, balance)>
  <!ATTLIST ACCOUNT
    accid ID,
    ownedby IDREFS>
]

```

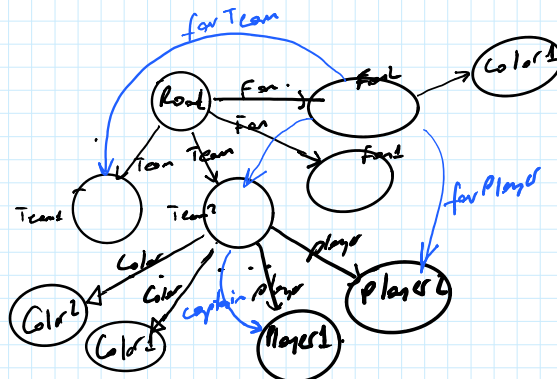
3.3) SPORTS DB:



```

<!DOCTYPE SPORT [
  <ELEMENT SPORT (TEAM*, FAN*)>
]

```



```

<!ELEMENT SPORT (TEAM*, FAN*)>
<!ELEMENT TEAM (NAME, PLAYER*, COLOR*)
  <!--
    <!-- ATTLIST TEAM
      teamID ID *REF
      captain IDREF *REF -->
  <!--
  <!-- ELEMENT PLAYER (NAME) -->
    <!-- ATTLIST PLAYER
      playerID ID *REF -->
  <!--
  <!-- ELEMENT FAN (NAME, COLOR*)
    <!-- ATTLIST FAN
      favPlayer IDREFS "
      favTeam IDREFS " -->
  <!--
  <!-- ELEMENT COLOR (*PCDATA)
  <!-- ELEMENT NAME (*PCDATA)
-->

```

If we put Player here, there will be many references from Team to players. We'll put all players inside the belonging Team.

There is no need to put "player" IDREF

3.5) RELATIONNAME (att-list)

```

<!DOCTYPE RelationName[
  <!-- ELEMENT RelationName (row*) -->
  <!-- ELEMENT row (att-list) -->
  for each att-list.
    <!-- ELEMENT att-name (*PCDATA) -->
-->

```

XML Schema:

- an alternative way to provide schema for XML docs.
- more powerful than DTD.
- Some extra capabilities:
 - ↳ allow * of occurrences of subelements
 - ↳ type declarations for single elements.
 - ↳ simple/complex type definitions. (Elements, attributes have types)
 - ↳ key, foreign key capability.

```

<? xml version = "1.0" encoding = "utf-8" ?>
<xs:schema xmlns:xs="http://www.w3.org/2001/XMLSchema">
  ...
</xs:schema>

```

→ This is XML doc.

...
</xs:schema>

name of namespace.

root tag = xs:schema

This is interpreted as "schema" in the xs namespace.

- most important tags =

{
xs:element
xs:complexType
xs:attribute
xs:simpleType
xs:key
xs:keyref
}

ELEMENTS:

- similar to element def. in DTD.

<xs:element name = "element name" type = "elem. type">
const's ... other stuff
</xs:element>

an element in the schema being defined.

{
simple = xs:integer
 xs:string
 xs:boolean
or
Complex
}

<xs:element name = "Title" type = "xs:string" />
<xs:element name = "Year" type = "xs:integer" />

<Title> </Title>
<Year> 2000 </Year> } valid XML

COMPLEX TYPES:

- may have several forms. One is sequence of elements.

1. way:
{
<xs:complexType name = "type name">
 <xs:sequence>
 list of elements
 </xs:sequence>
</xs:complexType>
}

2. way:
{
<xs:element name = "...">
 <xs:complexType>
 <xs:sequence>
 ...
 </xs:sequence>
 </xs:complexType>
</xs:element>
}

1) <? xml version = "1.0" encoding = "utf-8" ?>
2) <xs:schema xmlns:xs = "http://www.w3.org/2001/XMLSchema">
3) <xs:complexType name = "movieType">
4) <xs:sequence>
5) <xs:element name = "Title" type = "xs:string" />
6) <xs:element name = "Year" type = "xs:integer" />
7) </xs:sequence>

default
on occurrence.

<!doctype movies[

```

1) <?xml version="1.0" encoding="utf-8"?>
2) <xs:schema xmlns:xs="http://www.w3.org/2001/XMLSchema"
3)   <xs:complexType name="movieType">
4)     <xs:sequence>
5)       <xs:element name="Title" type="xs:string" />
6)       <xs:element name="Year" type="xs:integer" />
7)     </xs:sequence>
8)   </xs:complexType>
9)   <xs:element name="Movies">
10)    <xs:complexType>
11)      <xs:sequence>
12)        <xs:element name="Movie" type="movieType"
13)          minOccurs="0" maxOccurs="unbounded" />
14)      </xs:sequence>
15)    </xs:complexType>
16)  </xs:element>
17) </xs:schema>

```

Figure 11.12: A schema for movies in XML Schema

on occurrence.

<!DOCTYPE MOVIES[
 <ELEMENT MOVIES(MOVIE*)
 <ELEMENT MOVIE(TITLE, Year)
 <!-- Title (CDATA)
 <!-- Year (CDATA)>

- alternatives to xs:sequence:

↳ <xs:all
 ↳ <xs:choice

- ATTRIBUTES:

- Complex types can have attributes.

<xs:attribute name="..." title="xs:string"
 use="required"
 default="0" />

```

1) <?xml version="1.0" encoding="utf-8"?>
2) <xs:schema xmlns:xs="http://www.w3.org/2001/XMLSchema"
3)   <xs:complexType name="movieType">
4)     <xs:attribute name="title" type="xs:string"
5)       use="required" />
6)     <xs:attribute name="year" type="xs:integer"
7)       use="required" />
8)   </xs:complexType>
9)   <xs:element name="Movies">
10)    <xs:complexType>
11)      <xs:sequence>
12)        <xs:element name="Movie" type="movieType"
13)          minOccurs="0" maxOccurs="unbounded" />
14)      </xs:sequence>
15)    </xs:complexType>
16)  </xs:element>
17) </xs:schema>

```

Figure 11.14: Using attributes in place of simple elements

<!DOCTYPE MOVIES[
 <ELEMENT MOVIES(MOVIE*)
 <ELEMENT MOVIE EMPTY
 <!-- TITLE MOVIE
 title CDATA \$required
 year CDATA \$required>

>>

- RESTRICTED SIMPLE TYPES:

```

<xs:simpleType name="type name">
  <xs:restriction base="base type">
    upper and/or lower bounds
  </xs:restriction>
</xs:simpleType>

```

primitive types
 xs:integer...

Figure 11.16: Form of a range restriction

```

<xs:simpleType name="movieYearType">
  <xs:restriction base="xs:integer">
    <xs:minInclusive value="1915" />
  </xs:restriction>
</xs:simpleType>

```

Figure 11.17: A type that restricts integer values to be 1915 or greater

```

<xs:simpleType name="genreType">
  <xs:restriction base="xs:string">
    <xs:enumeration value="comedy" />
    <xs:enumeration value="drama" />
    <xs:enumeration value="scifi" />
    <xs:enumeration value="teen" />
  </xs:restriction>
</xs:simpleType>

```

Figure 11.18: An enumerated type in XML Schema

- KEYS:

- when we look at certain class C of elements → <xs:selector...
 fields within those elements will be key. → <xs:field...

<xs:key name="key-name"> will be used in FK if exist.
 <xs:selector xpath="..."> sequence of elements.
 <xs:field xpath="...">

`<xs:selector xpath = "...>` } sequence of elements.
`<xs:field xpath = "...>` } "subelement" or "attribute" of last element on the selector path:
 if attribute use @

```

1) <?xml version="1.0" encoding="utf-8" ?>
2) <xs:schema xmlns:xs="http://www.w3.org/2001/XMLSchema">
3) <xs:simpleType name="genreType">
4) <xs:restriction base="xs:string">
5) <xs:enumeration value="comedy" />
6) <xs:enumeration value="drama" />
7) <xs:enumeration value="sciFi" />
8) <xs:enumeration value="teen" />
9) </xs:restriction>
10) </xs:simpleType>
11) <xs:complexType name="movieType">
12) <xs:sequence>
13) <xs:element name="Title" type="xs:string" />
14) <xs:element name="Year" type="xs:integer" />
15) <xs:element name="Genre" type="genreType"
    minOccurs="0" maxOccurs="1" />
16) </xs:sequence>
17) </xs:complexType>
18) <xs:element name="Movies">
19) <xs:complexType>
20) <xs:sequence>
21) <xs:element name="Movie" type="movieType"
    minOccurs="0" maxOccurs="unbounded" />
22) </xs:sequence>
23) </xs:complexType>
24) <xs:key name="movieKey">
25) <xs:selector xpath="Movie" />
26) <xs:field xpath="Title" />
27) <xs:field xpath="Year" />
28) </xs:key>
29) </xs:element>
30) </xs:schema>
  
```

default is 1 occurrence.

key-alternative is unique.

together become "key".
those field should exist.

Figure 11.19: A schema for movies in XML Schema

FOREIGN KEYS:

references in XML Schema are to particular types of elements.

`<xs:keyref name="fk-name" refer="key-name">`

`<xs:selector xpath = "...>`

`<xs:field xpath = "...>`

`</xs:keyref>`

```

1) <?xml version="1.0" encoding="utf-8" ?>
2) <xs:schema xmlns:xs="http://www.w3.org/2001/XMLSchema">
3) <xs:element name="Stars">
4) <xs:complexType>
5) <xs:sequence>
6) <xs:element name="Star" minOccurs="1"
    maxOccurs="unbounded">
7) <xs:complexType>
8) <xs:sequence>
9) <xs:element name="Name"
    type="xs:string" />
10) <xs:element name="Address"
    type="xs:string" />
11) <xs:element name="StarredIn"
    minOccurs="0"
    maxOccurs="unbounded">
12) <xs:complexType>
13) <xs:attribute name="title"
    type="xs:string" />
14) <xs:attribute name="year"
    type="xs:integer" />
15) </xs:complexType>
16) </xs:element>
17) </xs:sequence>
18) </xs:complexType>
19) </xs:element>
20) </xs:sequence>
21) </xs:complexType>
22) <xs:keyref name="movieRef" refer="movieKey">
23) <xs:selector xpath="Star/StarredIn" />
24) <xs:field xpath="@title" />
25) <xs:field xpath="@year" />
26) </xs:keyref>
27) </xs:element>
  
```

each complex type is defined within the element that uses it.

starredIn element has 2 attributes which will be defined as fk.

Figure 11.20: Stars with a foreign key

Exercise 11.4.1: Give an example of a document that conforms to the XML Schema definition of Fig. 11.12 and an example of one that has all the elements mentioned, but does not conform to the definition.

Exercise 11.4.2: Rewrite Fig. 11.12 so that there is a named complex type for Movies, but no named type for Movie.

Exercise 11.4.3: Write the XML Schema definitions of Fig. 11.19 and 11.20 as a DTD.

```

1) <? xml version = "1.0" encoding = "utf-8" ?>
2) <xs:schema xmlns:xs = "http://www.w3.org/2001/XMLSchema">
3)   <xs:simpleType name = "genreType">
4)     <xs:restriction base = "xs:string">
5)       <xs:enumeration value = "comedy" />
6)       <xs:enumeration value = "drama" />
7)       <xs:enumeration value = "sciFi" />
8)       <xs:enumeration value = "teen" />
9)     </xs:restriction>
10)   </xs:simpleType>
11)   <xs:complexType name = "movieType">
12)     <xs:sequence>
13)       <xs:element name = "Title" type = "xs:string" />
14)       <xs:element name = "Year" type = "xs:integer" />
15)       <xs:element name = "Genre" type = "genreType"
16)         minOccurs = "0" maxOccurs = "1" />
17)     </xs:sequence>
18)   </xs:complexType>
19)   <xs:element name = "Movies">
20)     <xs:complexType>
21)       <xs:sequence>
22)         <xs:element name = "Movie" type = "movieType"
23)           minOccurs = "0" maxOccurs = "unbounded" />
24)       </xs:sequence>
25)     </xs:complexType>
26)     <xs:key name = "movieKey">
27)       <xs:selector xpath = "Movie" />
28)       <xs:field xpath = "Title" />
29)     </xs:key>
30) </xs:schema>

```

<!DOCTYPE MOVIES[
 <ELEMENT MOVIES(MOVIE*)>
 <ELEMENT MOVIE(Title,Year,Genre?)>
 <ELEMENT TITLE(TEXT)>
 < " Year(TEXT)>
 < " Genre(com|dra|sci|teen)>
]>

Figure 11.19: A schema for movies in XML Schema